

IoT Course — Hands-On #3

MQTT Publisher & Subscriber — Assignment #1

Student: Yakira Siboni

Broker: broker.hivemq.com | Topic: iot/home_YY/sensor_9426

1. Overview

This report documents the implementation and testing of an MQTT publisher and subscriber client using the paho-mqtt Python library. The goal is to understand how MQTT settings — specifically clean session flag, QoS level, and retained message flag — affect message delivery behavior.

Broker: broker.hivemq.com | Port: 1883 | Keep-alive: 90 seconds

Topic: iot/home_YY/sensor_9426

2. Client Code

2.1 Publisher Client

The publisher client connects to the broker, sets a Last Will message, and publishes a retained message to the topic. Key configurations: Client ID "IoT_YY_9426", keepalive=90, Last Will message defined, retained flag set to True on publish.

```
import paho.mqtt.client as mqtt #import the client1
import time
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# broker list
brokers=["iot.eclipse.org","broker.hivemq.com",\
        | "test.mosquitto.org","192.168.8.167","139.162.222.115"]

broker=brokers[1]

def on_log(client, userdata, level, buf):
    print("log: "+buf)
def on_connect(client, userdata, flags, rc):
    if rc==0:
        print("connected OK")
    else:
        print("Bad connection Returned code=",rc)
def on_disconnect(client, userdata, flags, rc=0):
    print("DisConnected result code "+str(rc))
def on_message(client,userdata,msg):
    topic=msg.topic
    m_decode=str(msg.payload.decode("utf-8","ignore"))
    print("message received",m_decode)

# Student: Yakira Siboni, ID: 208499426
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1, "IoT_YY_9426", clean_session=False) # create new client instance

client.on_connect=on_connect #bind call back function
client.on_disconnect=on_disconnect
client.on_log=on_log
client.on_message=on_message

# Last Will message - sent by broker if client disconnects unexpectedly
client.will_set("iot/home_YY/sensor_9426", payload="Publisher disconnected unexpectedly", qos=0, retain=False)

print("Connecting to broker ", broker)
port=1883
# keepalive set to 90 seconds as required
client.connect(broker, port, keepalive=90)

pub_topic = "iot/home_YY/sensor_9426"

# Publish a retained message to the topic
client.publish(pub_topic, "my 'retained' test message - Yakira Siboni 208499426", 0, True)

time.sleep(1)

print("End publish_client run script")

client.disconnect() # disconnect
```

Figure 1: mqtt_publisher_client_modified.py

2.2 Subscriber Client

The subscriber client connects to the broker, subscribes to the topic, and listens for 10 seconds before disconnecting. Key configurations: Client ID "IoT_sub_YY_9426", keepalive=90, loop_start() for asynchronous message reception.

```
import paho.mqtt.client as mqtt #import the client1
import time
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# broker list
brokers=["iot.eclipse.org", "broker.hivemq.com", \
        | "test.mosquitto.org", "192.168.8.167", "139.162.222.115"]

broker=brokers[1]

def on_log(client, userdata, level, buf):
    print("log: "+buf)
def on_connect(client, userdata, flags, rc):
    if rc==0:
        print("connected OK")
    else:
        print("Bad connection Returned code=",rc)
def on_disconnect(client, userdata, flags, rc=0):
    print("DisConnected result code "+str(rc))
def on_message(client,userdata,msg):
    topic=msg.topic
    m_decode=str(msg.payload.decode("utf-8","ignore"))
    print("Message Topic: ", topic)
    print("message received: ", m_decode)

# Student: Yakira Siboni, ID: 208499426
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1, "IoT_sub_YY_9426", clean_session=False) # create new client instance

client.on_connect=on_connect #bind call back function
client.on_disconnect=on_disconnect
client.on_log=on_log
client.on_message=on_message

print("Connecting to broker ", broker)
port=1883
# keepalive set to 90 seconds as required
client.connect(broker, port, keepalive=90)

sub_topic = 'iot/home_YY/sensor_9426'

client.loop_start() # Start loop
client.subscribe(sub_topic, qos=1)

time.sleep(10)
client.loop_stop() # Stop loop
client.disconnect()
```

Figure 2: mqtt_subscriber_client_modified.py

3. Test Results

For each test, the subscriber was run first, followed by the publisher. The subscriber script connects, subscribes to the topic, and listens for 10 seconds. The publisher connects, publishes a retained message, then disconnects.

Test 1 — clean_session=True, QoS=0

Settings: clean_session=True | Publish QoS=0 | Subscribe QoS=0 | Retain=True

Expected Result: No messages received (broker should not store messages)

Actual Result: Message received — due to retained message flag (r1 in log)

Explanation: Although clean_session=True and QoS=0 would normally result in no stored messages, the retained flag (r1) caused the broker to store the last published message and deliver it immediately upon subscription. The CONNECT log shows c1 (clean session on) and q0 (QoS 0). The retained message mechanism is independent from session persistence.

```
$ python mqtt_subscriber_client_modified.py
Connecting to broker broker.hivemq.com
log: Sending CONNECT (u0, p0, wr0, wq0, wf0, c1, k90) client_id=b'IoT_sub_YY_9426'
log: Sending SUBSCRIBE (d0, m1) [(b'iot/home_YY/sensor_9426', 0)]
log: Received CONNACK (0, 0)
connected OK
log: Received SUBACK
log: Received PUBLISH (d0, q0, r1, m0), 'iot/home_YY/sensor_9426', ... (52 bytes)
Message Topic: iot/home_YY/sensor_9426
message received: my 'retained' test message - Yakira Siboni 208499426
log: Sending DISCONNECT
Disconnected result code 0
```

Figure 3: Test 1 — Subscriber output

```
$ python mqtt_publisher_client_modified.py
Connecting to broker broker.hivemq.com
log: Sending CONNECT (u0, p0, wr0, wq0, wf1, c1, k90) client_id=b'IoT_YY_9426'
log: Sending PUBLISH (d0, q0, r1, m1), 'b'iot/home_YY/sensor_9426'', ... (52 bytes)
End publish_client run script
log: Sending DISCONNECT
Disconnected result code 0
```

Figure 4: Test 1 — Publisher output

Test 2 — clean_session=False, QoS=0

Settings: clean_session=False | Publish QoS=0 | Subscribe QoS=0 | Retain=True

Expected Result: Messages to be received (persistent session)

Actual Result: Message received — due to retained message flag (r1)

Explanation: The CONNECT log shows c0 confirming clean_session=False (persistent session). However, since Publish QoS=0, the broker does not queue messages for offline clients — the message was received again due to the retained flag, not session persistence. Note from the assignment: persistent session requires QoS >= 1 to fully guarantee delivery of missed messages.

```
$ python mqtt_subscriber_client_modified.py
Connecting to broker broker.hivemq.com
log: Sending CONNECT (u0, p0, wr0, wq0, wf0, c0, k90) client_id=b'IoT_sub_YY_9426'
log: Sending SUBSCRIBE (d0, m1) [(b'iot/home_YY/sensor_9426', 0)]
log: Received CONNACK (0, 0)
connected OK
log: Received SUBACK
log: Received PUBLISH (d0, q0, r1, m0), 'iot/home_YY/sensor_9426', ... (52 bytes)
Message Topic: iot/home_YY/sensor_9426
message received: my 'retained' test message - Yakira Siboni 208499426
log: Sending DISCONNECT
Disconnected result code 0
```

Figure 5: Test 2 — Subscriber output

```
$ python mqtt_publisher_client_modified.py
Connecting to broker broker.hivemq.com
log: Sending CONNECT (u0, p0, wr0, wq0, wf1, c0, k90) client_id=b'IoT_YY_9426'
log: Sending PUBLISH (d0, q0, r1, m1), 'b'iot/home_YY/sensor_9426'', ... (52 bytes)
End publish_client run script
log: Sending DISCONNECT
Disconnected result code 0
```

Figure 6: Test 2 — Publisher output

Test 3 — clean_session=True, QoS=1

Settings: clean_session=True | Publish QoS=1 | Subscribe QoS=1 | Retain=True

Expected Result: No messages received when reconnected (clean session clears state)

Actual Result: Message received — due to retained message flag (r1)

Explanation: The CONNECT log confirms c1 (clean session) and q1 (QoS=1 on publish). Despite clean_session=True clearing the session state on reconnect, the retained message was still delivered because retained messages are stored at the topic level by the broker — not in the client session. This demonstrates that retained messages and session persistence are two independent mechanisms.

```
$ python mqtt_subscriber_client_modified.py
Connecting to broker broker.hivemq.com
log: Sending CONNECT (u0, p0, wr0, wq0, wf0, c1, k90) client_id=b'IoT_sub_YY_9426'
log: Sending SUBSCRIBE (d0, m1) [(b'iot/home_YY/sensor_9426', 1)]
log: Received CONNACK (0, 0)
connected OK
log: Received SUBACK
log: Received PUBLISH (d0, q0, r1, m0), 'iot/home_YY/sensor_9426', ... (52 bytes)
Message Topic: iot/home_YY/sensor_9426
message received: my 'retained' test message - Yakira Siboni 208499426
log: Sending DISCONNECT
Disconnected result code 0
```

Figure 7: Test 3 — Subscriber output

```
$ python mqtt_publisher_client_modified.py
Connecting to broker broker.hivemq.com
log: Sending CONNECT (u0, p0, wr0, wq0, wf1, c1, k90) client_id=b'IoT_YY_9426'
log: Sending PUBLISH (d0, q1, r1, m1), 'b'iot/home_YY/sensor_9426'', ... (52 bytes)
End publish_client run script
log: Sending DISCONNECT
Disconnected result code 0
```

Figure 8: Test 3 — Publisher output

Test 4 — clean_session=False, QoS=1

Settings: clean_session=False | Publish QoS=1 | Subscribe QoS=1 | Retain=True

Expected Result: Messages received when reconnected (persistent session + reliable delivery)

Actual Result: Message received with PUBACK acknowledgement

Explanation: This is the correct combination for guaranteed delivery. The CONNECT log shows c0 (persistent session) and q1 (QoS=1). A notable new event appeared: "Sending PUBACK (Mid: 51)" — the subscriber sent an acknowledgement back to the broker confirming receipt. This is the QoS=1 handshake in action, ensuring the broker knows the message was delivered. This combination (clean_session=False + QoS >= 1) is the recommended setup for reliable IoT messaging.

```
$ python mqtt_subscriber_client_modified.py
Connecting to broker broker.hivemq.com
log: Sending CONNECT (u0, p0, wr0, wq0, wf0, c0, k90) client_id=b'IoT_sub_YY_9426'
log: Sending SUBSCRIBE (d0, m1) [(b'iot/home_YY/sensor_9426', 1)]
log: Received CONNACK (0, 0)
connected OK
log: Received SUBACK
log: Received PUBLISH (d0, q1, r1, m51), 'iot/home_YY/sensor_9426', ... (52 bytes)
Message Topic: iot/home_YY/sensor_9426
message received: my 'retained' test message - Yakira Siboni 208499426
log: Sending PUBACK (Mid: 51)
log: Sending DISCONNECT
Disconnected result code 0
```

Figure 9: Test 4 — Subscriber output

```
$ python mqtt_publisher_client_modified.py
Connecting to broker broker.hivemq.com
log: Sending CONNECT (u0, p0, wr0, wq0, wf1, c0, k90) client_id=b'IoT_YY_9426'
log: Sending PUBLISH (d0, q1, r1, m1), 'b'iot/home_YY/sensor_9426'', ... (52 bytes)
End publish_client run script
log: Sending DISCONNECT
Disconnected result code 0
```

Figure 10: Test 4 — Publisher output

Test 5 — clean_session=False, Publish QoS=0, Subscribe QoS=1

Settings: clean_session=False | Publish QoS=0 | Subscribe QoS=1 | Retain=True

Expected Result: No guaranteed delivery (publish QoS=0 means broker won't queue messages)

Actual Result: Message received TWICE — from two different sources

Explanation: This was the most interesting test. The subscriber received the message twice:

1. First delivery (r0, m1): A queued message from the persistent session — this was leftover from Test 4's QoS=1 publish, stored by the broker for the persistent session.
2. Second delivery (r1, m51): The retained message delivered upon subscription as usual.

Additionally, CONNACK(1,0) was received — the "1" indicates the broker recognized and resumed the existing persistent session. Both deliveries triggered PUBACK acknowledgements (Mid:1 and Mid:51). Key insight: subscribe QoS=1 alone is not enough for guaranteed delivery of new messages — the publish side must also use QoS >= 1 for the broker to queue messages for offline clients.

```
$ python mqtt_subscriber_client_modified.py
Connecting to broker broker.hivemq.com
log: Sending CONNECT (u0, p0, wr0, wq0, wf0, c0, k90) client_id=b'IoT_sub_YY_9426'
log: Sending SUBSCRIBE (d0, m1) [(b'iot/home_YY/sensor_9426', 1)]
log: Received CONNACK (1, 0)
connected OK
log: Received PUBLISH (d0, q1, r0, m1), 'iot/home_YY/sensor_9426', ... (52 bytes)
Message Topic: iot/home_YY/sensor_9426
message received: my 'retained' test message - Yakira Siboni 208499426
log: Sending PUBACK (Mid: 1)
log: Received SUBACK
log: Received PUBLISH (d0, q1, r1, m51), 'iot/home_YY/sensor_9426', ... (52 bytes)
Message Topic: iot/home_YY/sensor_9426
message received: my 'retained' test message - Yakira Siboni 208499426
log: Sending PUBACK (Mid: 51)
log: Sending DISCONNECT
Disconnected result code 0
```

Figure 11: Test 5 — Subscriber output

```
$ python mqtt_publisher_client_modified.py
Connecting to broker broker.hivemq.com
log: Sending CONNECT (u0, p0, wr0, wq0, wf1, c0, k90) client_id=b'IoT_YY_9426'
log: Sending PUBLISH (d0, q0, r1, m1), 'b'iot/home_YY/sensor_9426'', ... (52 bytes)
End publish_client run script
log: Sending DISCONNECT
Disconnected result code 0
```

Figure 12: Test 5 — Publisher output

4. Summary Table

The table below summarizes how Clean Session Flag, Retain Flag, Subscribe QoS, and Publish QoS affect whether published messages are always received by the subscriber.

Clean Session	Retain Flag	Subscribe QoS	Publish QoS	Msg Always Received?
true	false	0	0	No
true	false	0	1	No
true	false	1	0	No
true	false	1	1	No
false	false	0	0	No
false	false	0	1	No
false	false	1	0	No
false	false	1	1	Yes
true	true	0	0	Yes
true	true	0	1	Yes
true	true	1	0	Yes
true	true	1	1	Yes
false	true	0	0	Yes
false	true	0	1	Yes
false	true	1	0	Yes
false	true	1	1	Yes

Key takeaways:

- Retain Flag = True guarantees message delivery regardless of other settings, as the broker stores the last message at the topic level and delivers it on every new subscription.
- Guaranteed persistent delivery (without retain) requires both clean_session=False AND QoS ≥ 1 on both publish and subscribe sides.
- QoS level on publish determines whether the broker queues messages for offline clients. Subscribe QoS alone is not sufficient.